



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційні систем та технологій

Лабораторна робота № 7
із дисципліни «Технології розробки програмного забезпечення»
Тема: «Патерни проектування.»

Виконав
Студенти групи ІА-31:
Корнійчук М.Р

Перевірив:
Мягкий М.Ю

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

Варіант :

3. Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Посилання на гіт: https://github.com/MkEger/trpz-labs/tree/Lab_7

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Хід роботи

Короткий опис виконання роботи:

У рамках виконання лабораторної роботи було розроблено підсистему експорту документів для проєкту «Текстовий редактор» із використанням поведінкового патерну Template Method (Шаблонний метод). Метою реалізації було створення уніфікованого алгоритму генерації звітів, який гарантує правильну структуру

вихідного файлу (заголовок, метадані, контент, підвал), але дозволяє змінювати формат представлення даних (XML або LOG) без дублювання коду. Було створено ієрархію класів, де базовий клас визначає послідовність дій, а похідні класи реалізують специфічні кроки форматування.

Тема проекту: Текстовий редактор

1. Реалізувати не менше 3-х класів відповідно до обраної теми

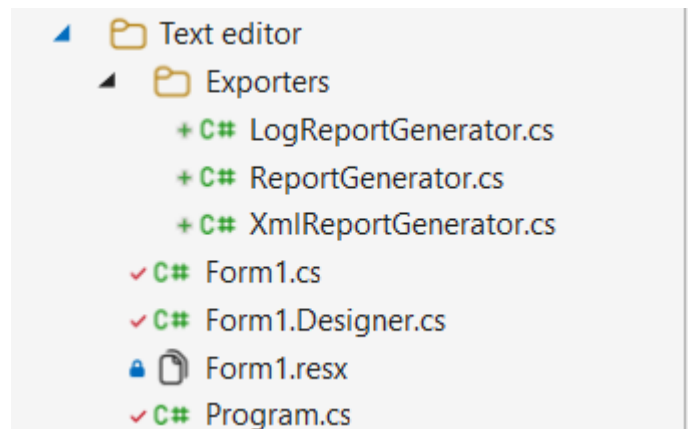


Рис. 1 - Структура проекту

У процесі виконання лабораторної роботи було створено три основні класи, які забезпечують роботу механізму експорту звітів. Наведемо детальний опис кожного класу:

1. ReportGenerator (Abstract Class) Абстрактний базовий клас, що визначає скелет алгоритму експорту.

- ExportDocument(string filePath, string content) – шаблонний метод, який містить незмінну логіку створення файлу. Він послідовно викликає методи генерації частин документа та записує результат на диск.
- Primitive Operations (CreateHeader, FormatBody, CreateFooter) – абстрактні методи, які мають бути реалізовані у підкласах для надання специфіки конкретного формату.
- Hook (FormatMetaData) – метод-гачок із базовою реалізацією, який може бути перевизначений у підкласах за потреби.

2. XmlReportGenerator (Concrete Class) Конкретна реалізація генератора для створення

структурованих XML-файлів. Функціонал:

- Реалізує методи форматування, додаючи відповідні XML-теги (<Report>, <Metadata>, <Body>).
- Здійснює базове екранування спеціальних символів у тексті для збереження валідності XML-структури.
- Перевизначає метод FormatMetaData для обгортання системної інформації у вкладені теги.

3. LogReportGenerator (Concrete Class) Конкретна реалізація генератора для створення текстових лог-файлів. Функціонал:

- Формує текстовий документ із використанням візуальних розділювачів (ліній, заголовків у верхньому регістрі).
- Додає до контенту маркери початку та кінця блоку даних.
- Генерує унікальний ідентифікатор (Checksum) у підвалі документа для контролю цілісності звіту.

2. Реалізувати один з розглянутих шаблонів за обраною темою

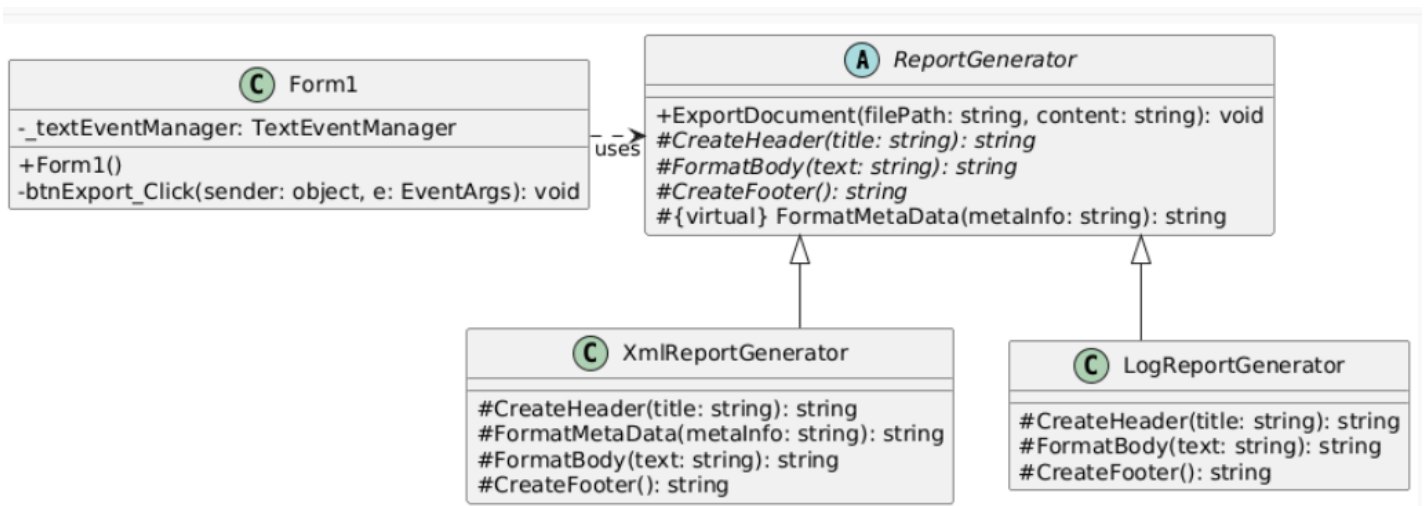


Рис. 2 - Діаграма класів

У рамках проєкту текстового редактора патерн Template Method було реалізовано для стандартизації процесу генерації звітів про стан документа.

Реалізація: Основна ідея полягала у винесенні інваріантної (незмінної) частини

алгоритму — послідовності формування блоків та запису файлу - у батьківський клас ReportGenerator. Варіативні частини (специфічний синтаксис формату) були делеговані підкласам.

Проблеми, які вирішує патерн:

- Уникнення дублювання: Логіка роботи з файловою системою (File IO) та збір системної статистики (дата, розмір) реалізовані в одному місці.
- Гарантія цілісності: Шаблонний метод гарантує, що кожен звіт матиме заголовок та підвал, і що метадані завжди будуть записані перед основним текстом.
- Розширюваність: Додавання нового формату (наприклад, CSV або JSON) вимагає лише реалізації методів форматування рядків, не зачіпаючи логіку збереження файлів.

Переваги використання:

- Повторне використання коду: Загальна логіка алгоритму (створення структури, запис файлу) реалізована один раз у базовому класі, що усуває дублювання (принцип DRY).
- Інверсія управління: Базовий клас контролює порядок виконання операцій, делегуючи підкласам лише реалізацію специфічних кроків.
- Надійність: Гарантується дотримання єдиної структури звіту (заголовок → метадані → тіло → підвал) для будь-якого формату.
- Легкість розширення: Додавання нового формату вимагає лише реалізації методів форматування рядків, не зачіпаючи логіку роботи з файловою системою.

Код:

```

1  using System;
2  using System.IO;
3  using System.Text;
4
5  namespace TextEditor.Exporters
6  {
7      public abstract class ReportGenerator
8      {
9          public void ExportDocument(string filePath, string content)
10         {
11             var sb = new StringBuilder();
12
13             sb.AppendLine(CreateHeader(filePath));
14
15             string sysInfo = $"Generated: {DateTime.Now} | Size: {content.Length} chars";
16             sb.AppendLine(FormatMetadata(sysInfo));
17
18             sb.AppendLine(FormatBody(content));
19
20             sb.AppendLine(CreateFooter());
21
22             File.WriteAllText(filePath, sb.ToString(), Encoding.UTF8);
23         }
24
25         protected abstract string CreateHeader(string title);
26         protected abstract string FormatBody(string text);
27         protected abstract string CreateFooter();
28
29         protected virtual string FormatMetadata(string metaInfo)
30         {
31             return $"[SYSTEM INFO] {metaInfo}";
32         }
33     }
34 }

```

Рис. 3.1 – ReportGenerator.cs

```

1  namespace TextEditor.Exporters
2  {
3      public class XmlReportGenerator : ReportGenerator
4      {
5          protected override string CreateHeader(string title)
6          {
7              return "<?xml version='1.0' encoding='utf-8'>\n<Report>";
8          }
9
10         protected override string FormatMetadata(string metaInfo)
11         {
12             return $" <Metadata>\n    <Info>{metaInfo}</Info>\n </Metadata>";
13         }
14
15         protected override string FormatBody(string text)
16         {
17             string safeText = text.Replace("<", "&lt;").Replace(">", "&gt;").Replace("&", "&amp;");
18             return $" <Body>\n    <Content>{safeText}</Content>\n </Body>";
19         }
20
21         protected override string CreateFooter()
22         {
23             return "</Report>";
24         }
25     }
26 }

```

Рис. 3.2 – XmlReportGenerator.cs

```

1      using System;
2
3      namespace TextEditor.Exporters
4      {
5          public class LogReportGenerator : ReportGenerator
6          {
7              protected override string CreateHeader(string title)
8              {
9                  string fileName = System.IO.Path.GetFileName(title);
10                 return "=====\n" +
11                     $"LOG REPORT: {fileName.ToUpper()}\n" +
12                     "=====";
13             }
14
15             protected override string FormatBody(string text)
16             {
17                 return $"\\n>>> BEGIN CONTENT >>>\\n{text}\\n<<< END CONTENT <<<";
18             }
19
20             protected override string CreateFooter()
21             {
22                 return "\\n-----\\n" +
23                     $"End of Report. Checksum: {Guid.NewGuid().ToString().Substring(0, 8)}\\n" +
24                     "-----";
25             }
26         }
27     }

```

Рис. 3.3 – LogReportGenerator.cs

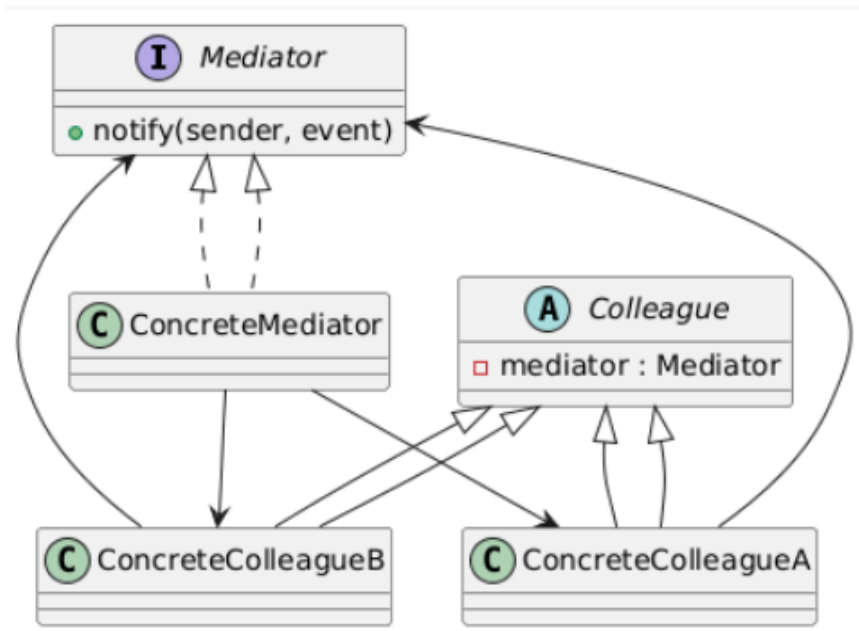
Висновок: У ході роботи було вдосконалено проєкт текстового редактора з використанням шаблону Template Method (Шаблонний метод). Реалізовано ієрархію класів генерації звітів: абстрактний клас ReportGenerator, що визначає структуру алгоритму створення звіту, та конкретні класи XmlReportGenerator і LogReportGenerator, які надають специфічну реалізацію кроків форматування даних. Застосування патерну дозволило уникнути дублювання коду при формуванні структури документа, забезпечило єдиний стандарт для всіх вихідних файлів та спростило розширення системи новими типами експорту.

Відповіді на контрольні питання:

1. Яке призначення шаблону «Посередник»?

Шаблон Посередник (Mediator) використовується для організації взаємодії між багатьма об'єктами через центральний об'єкт-посередник, щоб зменшити кількість прямих залежностей між ними. Це знижує зв'язність та спрощує розширення логіки.

2. Нарисуйте структуру шаблону «Посередник».



3. Які класи входять в шаблон «Посередник», та яка між ними взаємодія?

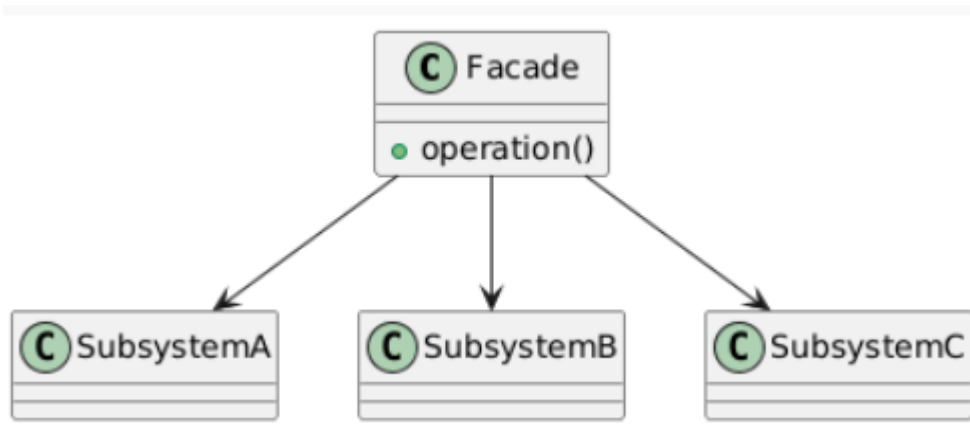
- Mediator - інтерфейс посередника
- ConcreteMediator - координує об'єкти
- Colleague - базовий учасник взаємодії
- ConcreteColleague - класи, що взаємодіють через посередника

Колеги не спілкуються напряму — вони викликають методи посередника.

4. Яке призначення шаблону «Фасад»?

Шаблон Фасад (Facade) надає спрощений інтерфейс до складної підсистеми, приховуючи внутрішні деталі реалізації й зменшуючи залежність клієнта від внутрішньої структури системи.

5. Нарисуйте структуру шаблону «Фасад»



6. Які класи входять в шаблон «Фасад», та яка між ними взаємодія?

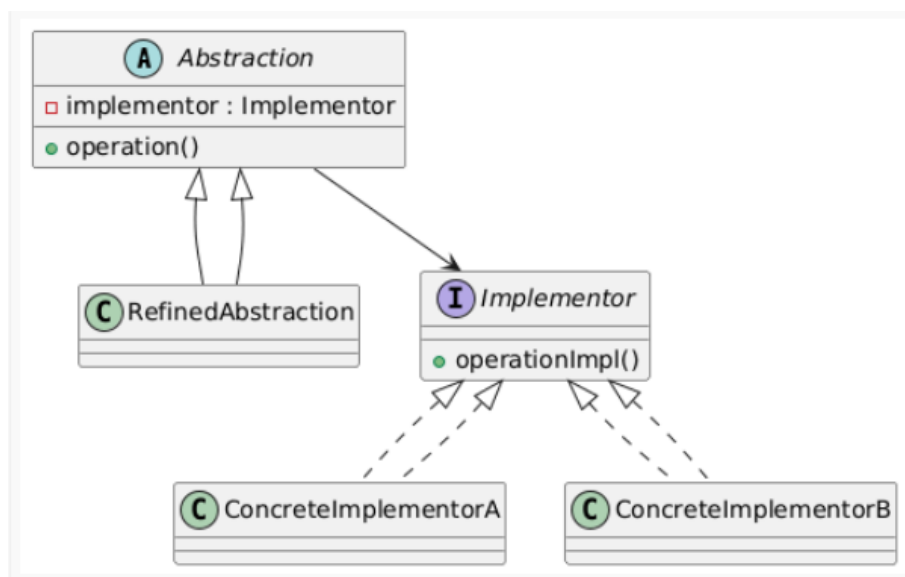
- Facade - єдина точка доступу, викликає підсистеми
- SubsystemA/B/C - внутрішні класи, з якими клієнт не працює напряду

Клієнт працює лише через фасад → зниження зв'язності.

7. Яке призначення шаблону «Міст»?

Шаблон Міст (Bridge) відділяє абстракцію від реалізації, дозволяючи їх змінювати незалежно. Замість жорсткого спадкування використовується композиція.

8. Нарисуйте структуру шаблону «Міст».



9. Які класи входять в шаблон «Міст», та яка між ними взаємодія?

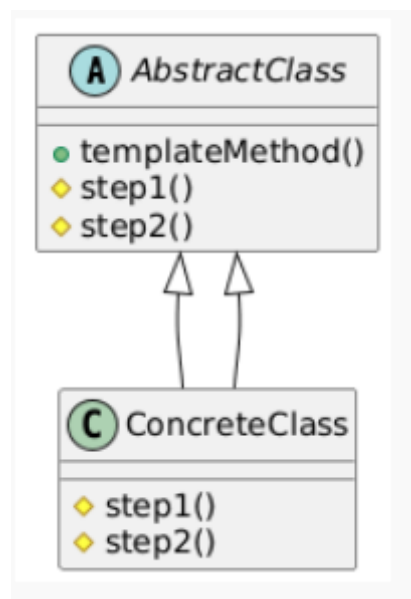
- Abstraction - високорівневий інтерфейс
- RefinedAbstraction - розширена версія абстракції
- Implementor - інтерфейс реалізації
- ConcreteImplementor - конкретні реалізації

Абстракція викликає реалізацію через композицію, а не спадкування.

10. Яке призначення шаблону «Шаблонний метод»?

Шаблон Template Method визначає скелет алгоритму у базовому класі, а підкласи можуть перевизначати окремі кроки без зміни структури алгоритму

11. Нарисуйте структуру шаблону «Шаблонний метод».



12. Які класи входять в шаблон «Шаблонний метод», та яка між ними взаємодія?

- AbstractClass - містить алгоритм та абстрактні методи
- ConcreteClass - реалізує кроки алгоритму

Клієнт викликає `templateMethod()`, а підкласи визначають поведінку деталей.

13. Чим відрізняється шаблон «Шаблонний метод» від «Фабричного методу»?

- Шаблонний метод контролює послідовність виконання алгоритму, дозволяючи змінювати окремі етапи.
- Фабричний метод управляє створенням об'єктів, делегуючи вибір класу підкласам.

14. Яку функціональність додає шаблон «Міст»?

Шаблон Міст додає можливість розширювати абстракцію та реалізацію незалежно одна від одної, уникаючи вибуху кількості класів при комбінації різних реалізацій.

Також він:

- зменшує зв'язність між рівнями системи
- спрощує зміну реалізації під час виконання
- підтримує платформозалежні реалізації (наприклад, різні драйвери)